Relatório de Testes de Software — Marketplace (marketplace_db)

Laboratório de Banco de Dados (UCB) — Projeto final · Grupo 9

Data da execução: 19/10/2026

1. Objetivo e escopo

Este relatório documenta os testes automatizados aplicados aos três scripts SQL do projeto Marketplace:

- 01_ddl.sql: criação do banco e das tabelas, com chaves, restrições (constraints) e regras de negócio.
- 02_carga.sql: carga de dados (45 usuários, 55 produtos, 50 pedidos, 151 itens de pedido, entre outros).
- 03_consultas.sql: 15 consultas SQL sobre o domínio.

O objetivo é verificar se a estrutura rejeita dados inválidos, se a carga é consistente com as regras de negócio (RN) e se cada consulta devolve o resultado esperado.

2. Ambiente de execução

- IDE: IntelliJ IDEA 2025.3.3, projeto Maven (marketplace-testes)
- JDK: Eclipse Temurin 17
- Framework de testes: JUnit 5 (junit-jupiter 5.10.2) com maven-surefire-plugin 3.2.5
- Acesso ao banco: JDBC com MySQL Connector/J 8.3.0
- Banco de dados: MySQL Server 8.0.46 (X64) local, Windows
- Os scripts ficam em src/test/resources/sql e os testes em src/test/java/marketplace

Testes de API: Postman (aplicativo desktop) com a coleção Mercado API - testes, executada contra o backend Mercado (Spring Boot 4.1, Java 21) em http://localhost:8081, conectado ao mesmo marketplace_db.

3. Estratégia de testes

Foram implementadas três classes de teste unitário (JUnit), cada uma focada em um script. O acesso ao banco é feito por uma classe auxiliar (DbHelper) que executa os scripts na ordem 01 e 02.

- DdlTest: recria apenas a estrutura (tabelas vazias) e tenta violar constraints, esperando que o banco lance exceção. Também verifica a exclusão em cascata.
- CargaTest: recria o banco com a carga completa e confere contagens de registros e regras de integridade (totais, preços, datas, status, papéis de usuário).
- ConsultasTest: executa as consultas lidas diretamente do arquivo 03_consultas.sql e compara o resultado com o esperado para a carga atual.

Uma quarta classe, BugsConhecidosTest, reúne testes que devem falhar: cada um descreve o comportamento correto esperado e falha porque o banco ou a consulta não o atendem, o que registra a evidência dos bugs da seção 6.

Os testes unitários do backend (UserServiceLoginTest) usam JUnit com Mockito e não dependem do banco, validando apenas a regra de login do UserService.

Os testes de API foram executados no Postman (coleção Mercado API - testes) contra o backend Mercado, em Spring Boot, que usa o mesmo banco marketplace_db. Os testes de interface (E2E) foram escritos em Playwright para o frontend entregue na versão atual do projeto (nove casos: catálogo, busca, login, cadastro, compra, pagamento e cancelamento), e foram executados no navegador Chromium contra o sistema real (backend Mercado e banco marketplace_db locais).

4. Casos de teste

4.1 Estrutura e constraints (DdlTest — 01_ddl.sql)

ID	Descrição	Passos para execução	Dados de entrada	Saída esperada	Critério de sucesso
DDL-U-01	Rejeitar e-mail duplicado (uq_user_email)	1) Inserir usuário 'a' com e-mail x@x.com. 2) Inserir usuário 'b' com o mesmo e-mail. 3) Capturar a exceção.	e-mail x@x.com repetido	SQLIntegrityConstraintViolationException	Exceção lançada; segundo usuário não inserido
DDL-U-02	Rejeitar produto com valor zero (ck_product_value_positive)	1) Criar usuário, vendedor e categoria. 2) Inserir produto com value = 0.	value = 0	SQLException por violação de CHECK	Exceção lançada; produto não inserido
DDL-U-03	Não excluir vendedor que possui produtos (RN02, ON DELETE RESTRICT)	1) Criar vendedor, categoria e produto. 2) Excluir o vendedor.	DELETE em tb_seller com produto vinculado	SQLException por violação de chave estrangeira	Exceção lançada; vendedor mantido
DDL-U-04	Rejeitar status de pedido inválido (RN15)	1) Criar cliente e pedido. 2) Inserir status 'xyz' em tb_order_status.	status = 'xyz'	SQLException por violação de CHECK	Exceção lançada; status não inserido
DDL-U-05	Exclusão em cascata de usuário para cliente (ON DELETE CASCADE)	1) Criar usuário e cliente. 2) Excluir o usuário. 3) Contar registros em tb_client.	DELETE em tb_user	tb_client com 0 registros	Contagem igual a 0

4.2 Carga de dados (CargaTest — 02_carga.sql)

ID	Descrição	Passos para execução	Dados de entrada	Saída esperada	Critério de sucesso
CARGA-U-01	Quantidade de registros por tabela	1) Recriar banco com DDL e carga. 2) Executar COUNT(*) em cada tabela.	Scripts 01 e 02	user 45, seller 20, client 30, category 14, supplier 16, product 55, product_supplier 57, cart 45, buyItem 74, order 50, orderItem 151, order_status 163	Todas as contagens iguais às esperadas
CARGA-U-02	RN13: total do pedido igual à soma dos itens	1) Comparar tb_order.totalvalue com SUM(quantity × unit_value) de tb_orderItem.	Carga completa	0 pedidos divergentes	Contagem de divergências igual a 0
CARGA-U-03	Total e quantidade do carrinho iguais aos itens	1) Comparar tb_cart.totalvalue e itemquantity com a soma de tb_buyItem.	Carga completa	0 carrinhos divergentes	Contagem de divergências igual a 0

ID	Descrição	Passos para execução	Dados de entrada	Saída esperada	Critério de sucesso
CARGA-U-04	RN11: valor unitário do item igual ao preço do produto	1) Juntar tb_orderItem com tb_product. 2) Contar itens com unit_value diferente de value.	Carga completa	0 itens divergentes	Contagem igual a 0
CARGA-U-05	RN17: nenhum pedido com data futura	1) Contar pedidos com date_order maior que a data atual.	Carga completa	0 pedidos	Contagem igual a 0
CARGA-U-06	Todo pedido possui histórico de status (RN14)	1) Contar pedidos sem nenhum registro em tb_order_status.	Carga completa	0 pedidos sem status	Contagem igual a 0
CARGA-U-07	RN01: usuários com os dois papéis (vendedor e cliente)	1) Contar usuários presentes em tb_seller e em tb_client.	Carga completa	5 usuários	Contagem igual a 5
CARGA-U-08	Nenhum produto sem vendedor ou categoria (RN02, RN04)	1) Juntar produto com vendedor e categoria. 2) Contar ausências.	Carga completa	0 produtos órfãos	Contagem igual a 0
CARGA-U-09	Casos de contorno propositais da carga	1) Contar produtos sem fornecedor. 2) Contar carrinhos vazios. 3) Contar produtos com estoque zero.	Carga completa	5 produtos sem fornecedor, 1 carrinho vazio, 4 produtos com estoque zero	As três contagens conferem

4.3 Consultas (ConsultasTest — 03_consultas.sql)

ID	Descrição	Passos para execução	Dados de entrada	Saída esperada	Critério de sucesso
CONS-U-01	Listar todos os produtos	Executar a consulta 1 e contar as linhas	—	55 linhas	55 linhas retornadas
CONS-U-02	Produtos acima de R$ 100, do mais caro ao mais barato	Executar a consulta 2; conferir contagem, primeiro item e ordenação	value > 100	37 linhas; primeiro: Notebook Acer Aspire 5 Pro; valores em ordem não crescente	Contagem, primeiro item e ordem corretos

ID	Descrição	Passos para execução	Dados de entrada	Saída esperada	Critério de sucesso
CONS-U-03	Produtos com 'Celular' no nome	Executar a consulta 3	LIKE '%Celular%'	6 linhas, todas com 'Celular' no nome	Contagem e conteúdo corretos
CONS-U-04	Produtos entre R$ 100 e R$ 2000 nas categorias 3 ou 4	Executar a consulta 4	id_category IN (3,4)	0 linhas (as categorias 3 e 4 são raízes sem produtos diretos)	Resultado vazio; ver achado B1
CONS-U-05	Fornecedores sem telefone	Executar a consulta 5	phone IS NULL	Alimentos Brasil, Utilcasa Distribuicao e Norte Suprimentos	Conjunto de nomes idêntico ao esperado
CONS-U-06	Categoria de cada produto	Executar a consulta 6	JOIN produto e categoria	55 linhas	55 linhas retornadas
CONS-U-07	Produto e usuário vendedor	Executar a consulta 7	JOIN produto, vendedor e usuário	55 linhas	55 linhas retornadas
CONS-U-08	Produtos com ou sem registro em pedidos	Executar a consulta 8; contar linhas e linhas sem pedido	LEFT JOIN com tb_orderItem	186 linhas (151 itens + 35 produtos sem pedido)	186 linhas, sendo 35 com id_order nulo
CONS-U-09	Pedidos com mais de um status	Executar a consulta 9	GROUP BY com HAVING COUNT(*) > 1	45 linhas, todas com quantidade maior que 1	Contagem e condição corretas
CONS-U-10	Quantidade de produtos por fornecedor	Executar a consulta 10	LEFT JOIN com tb_product_supplier	16 fornecedores; Felipe Distribuidora 17, Bebidas do Vale 17, Grao Dourado Alimentos 16, Casa Boa Suprimentos 2, Elias Comercio 0	Valores conferem
CONS-U-11	Produtos acima da média de preço	Executar a consulta 11	Subconsulta com AVG(value)	Média de 821,57; 17 produtos, todos acima da média	17 linhas, todas com valor > 821,57
CONS-U-12	Clientes que já fizeram pedido	Executar a consulta 12	EXISTS em tb_order	6 clientes: gabriela.nascimento, vinicius.teixeira, sabrina.santos, mariana.gomes, bruno.campos, fabio.nascimento	Conjunto idêntico ao esperado
CONS-U-13	Produtos nunca comprados	Executar a consulta 13	NOT EXISTS em tb_orderItem	35 linhas	35 linhas retornadas

ID	Descrição	Passos para execução	Dados de entrada	Saída esperada	Critério de sucesso
CONS-U-14	Pedido de maior valor	Executar a consulta 14	Subconsulta com MAX(totalvalue)	Pedido 4, R$ 19.476,86, cliente 36	Os três valores conferem
CONS-U-15	Status mais recente de cada pedido e cliente	Executar a consulta 15	MAX(date_status) por pedido	50 linhas; pedido 1 entregue, pedido 17 cancelado, pedido 9 aberto	Contagem e status conferem

4.4 Testes que evidenciam bugs (BugsConhecidosTest)

Estes quatro testes descrevem o comportamento correto esperado e falham de propósito, pois o banco ou a consulta não o atendem. Cada teste roda numa transação desfeita ao final, sem alterar a carga.

ID	Descrição	Passos para execução	Dados de entrada	Saída esperada	Critério de sucesso	Resultado obtido
BUG-U-01	A consulta 4 deve retornar produtos (achado B1)	1) Recriar o banco com DDL e carga. 2) Executar a consulta 4 lida do 03_consultas.sql. 3) Contar as linhas.	Valor entre R$ 100 e R$ 2000, id_category IN (3,4)	Pelo menos 1 linha	Contagem maior que 0	FALHOU: retornou 0 linhas (Expected true, Actual false). Figura 5
BUG-U-02	O banco deve rejeitar pedido com data futura (RN17, achado B4)	1) Inserir em tb_order um pedido com date_order daqui a 1 ano para o cliente 16. 2) Capturar a exceção.	date_order = agora + 1 ano, totalvalue 10	SQLException	Exceção lançada	FALHOU: nenhuma exceção lançada, o pedido foi aceito. Figura 6
BUG-U-03	O banco deve rejeitar categoria ligada a ela mesma (RN05, achado B4)	1) Inserir em tb_category a categoria 500 com id_category_parent = 500. 2) Capturar a exceção.	id_category 500, id_category_parent 500	SQLException	Exceção lançada	FALHOU: nenhuma exceção lançada, a categoria foi aceita. Figura 7

ID	Descrição	Passos para execução	Dados de entrada	Saída esperada	Critério de sucesso	Resultado obtido
BUG-U-04	O total do pedido deve bater com a soma dos itens (RN13, achado B4)	1) Inserir o pedido 900 com total R$ 1,00. 2) Inserir o item (produto 1, quantidade 2, R$ 100,00). 3) Contar pedidos com total divergente.	totalvalue 1,00 com itens somando 200,00	0 pedidos divergentes	Contagem igual a 0	FALHOU: 1 pedido divergente (Expected 0, Actual 1). Figura 8

4.5 Testes de API (Postman — backend Mercado)

Os testes de API usam a coleção Mercado API - testes (22 requisições e 47 verificações), executada no Postman contra o backend Mercado em http://localhost:8081, com o banco marketplace_db carregado. As requisições dependem umas das outras (por exemplo, o usuário criado no API-01 é usado no login, no carrinho e no pedido) e rodam em ordem. Os resultados esperados vieram da leitura do código do backend.

ID	Descrição	Requisição	Dados de entrada	Saída esperada	Resultado obtido
API-01	Cadastrar usuário válido	POST /users	username, e-mail e senha com 6 ou mais caracteres	201; id, username e e-mail; sem a senha	Aprovado (3/3)
API-02	Cadastrar com e-mail repetido	POST /users	e-mail já cadastrado	422, 'E-mail já cadastrado'	Aprovado (2/2)
API-03	Cadastrar com senha curta	POST /users	senha de 5 caracteres	400; erro no campo password	Aprovado (2/2)
API-04	Login com credenciais válidas	POST /login	e-mail e senha corretos	200; id e e-mail do usuário; sem a senha	Aprovado (3/3)
API-05	Login com senha incorreta	POST /login	senha errada	422, 'E-mail ou senha inválidos' (o plano previa 401)	Aprovado (2/2)
API-06	Listar produtos	GET /products	—	200; ao menos 55 produtos; existe produto com estoque	Aprovado (3/3)

ID	Descrição	Requisição	Dados de entrada	Saída esperada	Resultado obtido
API-07	Criar produto válido	POST /products	value 99.90, quantity 10, sellerId 1, categoryId 5	201; produto criado com os dados enviados	Aprovado (2/2)
API-08	Criar produto com valor zero	POST /products	value 0	400; erro no campo value	Aprovado (2/2)
API-09	Criar produto com vendedor inexistente	POST /products	sellerId 9999	404, 'Vendedor não encontrado: 9999'	Aprovado (2/2)
API-10	Buscar produto inexistente	GET /products/999999	—	404, 'Produto não encontrado: 999999'	Aprovado (2/2)
API-11	Excluir produto criado no teste	DELETE /products/{id}	produto criado no API-07	204	Aprovado (1/1)
API-12	Tornar usuário cliente	POST /users/{id}/client	corpo vazio	201 (cria o carrinho)	Aprovado (1/1)
API-13	Consultar carrinho do cliente	GET /clients/{id}/cart	—	200; carrinho vazio	Aprovado (2/2)
API-14	Fechar pedido com carrinho vazio	POST /carts/{id}/checkout	—	422, 'O carrinho está vazio'	Aprovado (2/2)
API-15	Adicionar item ao carrinho	POST /carts/{id}/items	produto com estoque, quantity 2	200; 1 item, quantidade 2, subtotal igual a valor unitário × 2	Aprovado (3/3)
API-16	Adicionar item acima do estoque	POST /carts/{id}/items	quantity 999999	422, 'Estoque insuficiente'	Aprovado (2/2)
API-17	Fechar pedido válido	POST /carts/{id}/checkout	—	201; status ABERTO; total igual a valor unitário × 2 (RN13)	Aprovado (3/3)
API-18	Estoque diminui após o pedido	GET /products/{id}	—	200; estoque reduzido em 2	Aprovado (2/2)
API-19	Status inválido (ABERTO para ENTREGUE)	PATCH /orders/{id}/status	status ENTREGUE	422, 'Não é possível ir de ABERTO para ENTREGUE'	Aprovado (2/2)

ID	Descrição	Requisição	Dados de entrada	Saída esperada	Resultado obtido
API-20	Status válido (ABERTO para PAGO)	PATCH /orders/{id}/status	status PAGO	200; status PAGO e histórico com 2 registros	Aprovado (2/2)
API-21	Cancelar pedido pago	PATCH /orders/{id}/status	status CANCELADO	200; status CANCELADO	FALHOU (0/2): status 200 e status CANCELADO não atendidos. Achado B5, Figura 10
API-22	Estoque volta após o cancelamento	GET /products/{id}	—	200; estoque igual ao inicial	FALHOU (1/2): estoque não voltou ao valor inicial, consequência do API-21. Figura 10

4.6 Teste unitário do login (JUnit + Mockito)

A classe UserServiceLoginTest testa o método UserService.login do backend Mercado sem banco e sem Spring: o repositório de usuários é simulado com Mockito e a senha usa BCrypt, como na aplicação. O sistema não usa token JWT, então os casos de teste unitário de login foram adaptados ao que existe: o login devolve id, nome e e-mail do usuário.

ID	Descrição	Dados de entrada	Saída esperada	Critério de sucesso	Resultado
LG-U-01	Login com credenciais válidas	maria@email.com / senha123 (usuário existente)	UserResponse com id 1, nome maria e e-mail maria@email.com	Os três campos conferem	Aprovado
LG-U-02	Login com senha incorreta	maria@email.com / errada	BusinessException "E-mail ou senha inválidos"	Exceção lançada com a mensagem esperada	Aprovado
LG-U-03	Login com e-mail inexistente	x@email.com / senha123	BusinessException "E-mail ou senha inválidos"	Mesma mensagem do caso de senha errada (não revela se o e-mail existe)	Aprovado
LG-U-04	Senha guardada com hash	Senha senha123 codificada por BCrypt	Hash diferente do texto puro, iniciado em $2, e que valida a senha	Senha nunca fica em texto puro	Aprovado

4.7 Testes de interface (E2E, Playwright)

Os testes E2E abrem o frontend (Nexa) em um navegador Chromium e reproduzem o que uma pessoa faria na tela, contra o backend Mercado e o banco marketplace_db reais. Cada teste cria o próprio usuário (e-mail único por execução) e não depende de senhas da carga. Os testes rodam em sequência, e cada um guarda um print da tela como evidência.

ID	Descrição	Passos principais	Resultado esperado	Resultado
E2E-01	Catálogo carrega os produtos da API	Abrir a página inicial	Cards de produto visíveis e contador de produtos	Aprovado
E2E-02	Busca filtra a lista	Digitar um termo inexistente e depois limpar	Mensagem "Nada por aqui"; ao limpar, a lista volta ao total	Aprovado
E2E-03	Adicionar ao carrinho sem login	Clicar em Adicionar sem estar logado	Janela pedindo para entrar na conta	Aprovado
E2E-04	Login com credenciais inválidas	Entrar com e-mail e senha inexistentes	Erro "E-mail ou senha inválidos"	Aprovado
E2E-05	Cadastro de usuário	Criar conta com nome, e-mail e senha	Usuário já logado, nome no topo	Aprovado
E2E-06	Login e logout	Entrar com a conta criada e sair	Nome no topo ao entrar; botão Entrar ao sair	Aprovado
E2E-07	Compra	Adicionar um produto, abrir o carrinho e finalizar	Pedido em Meus pedidos como "Aguardando pagamento"; carrinho zerado	Aprovado
E2E-08	Pagar o pedido	Em Meus pedidos, clicar em Pagar agora	Status Pago	Aprovado

ID	Descrição	Passos principais	Resultado esperado	Resultado
E2E-09	Cancelar um pedido novo	Finalizar um pedido e cancelar em seguida, confirmando	Status Cancelado	Falhou: o pedido continuou em "Aguardando pagamento" (achado B5)

5. Execução e resultados

Os testes de banco foram executados no IntelliJ IDEA, uma classe por vez, na ordem DdlTest, CargaTest, ConsultasTest e BugsConhecidosTest.

Classe	Testes	Aprovados	Falhas	Tempo
DdlTest	5	5	0	2 s 603 ms
CargaTest	9	9	0	100 ms
ConsultasTest	15	15	0	83 ms
BugsConhecidosTest	4	0	4 (esperadas)	90 ms
UserServiceLoginTest (unitário, login)	4	4	0	1 s 693 ms
Postman (API, 22 requisições)	47	44	3	3 s 513 ms
Playwright (E2E, interface)	9	8	1	22 s 600 ms
Total	93	85	8	—

Evidências

Figura 1 — Execução do DdlTest: 5 testes aprovados.

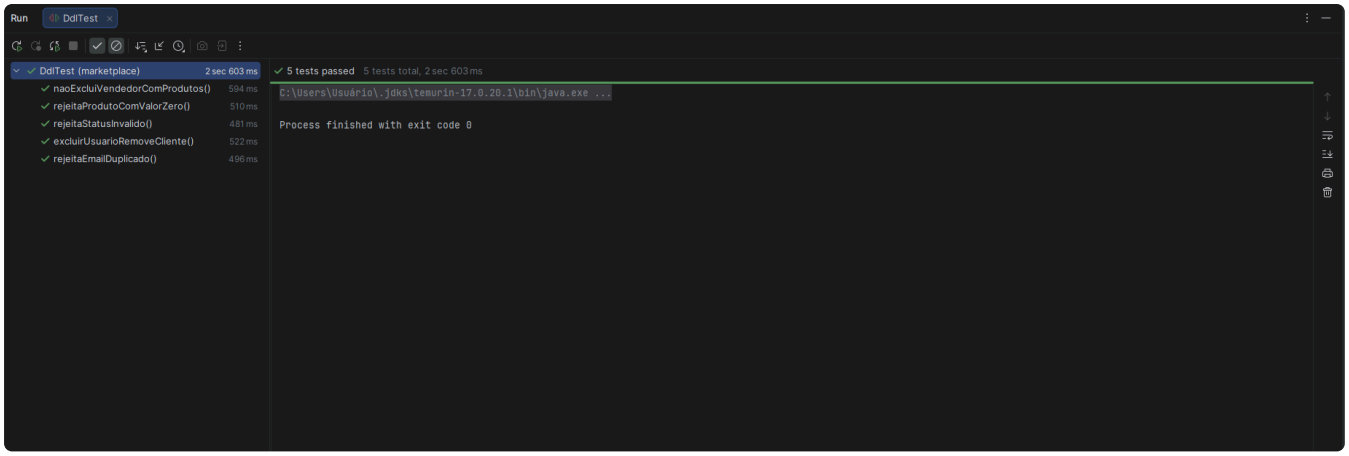


Figura 2 — Execução do CargaTest: 9 testes aprovados.

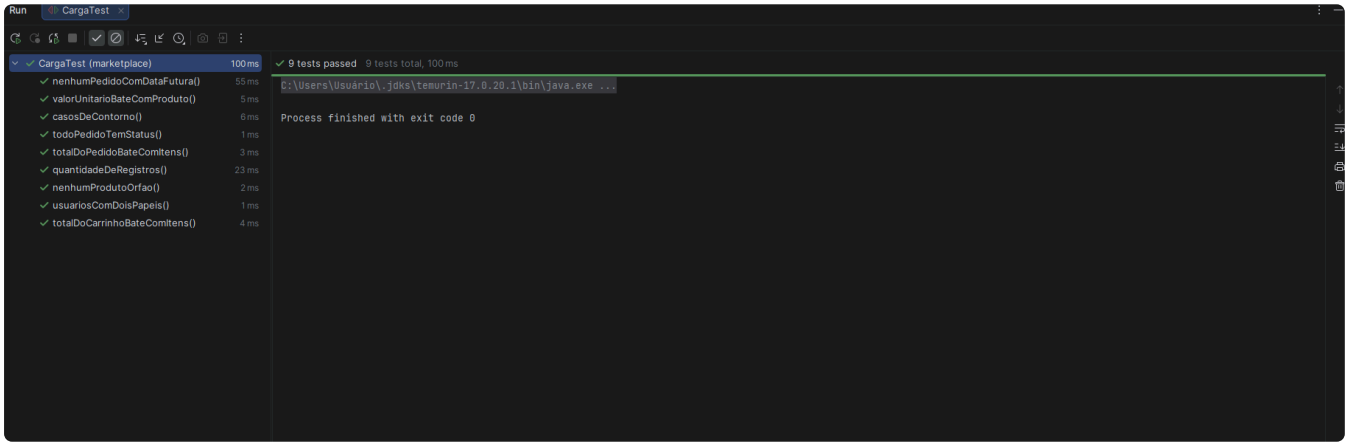


Figura 3 — Execução do ConsultasTest: 15 testes aprovados.

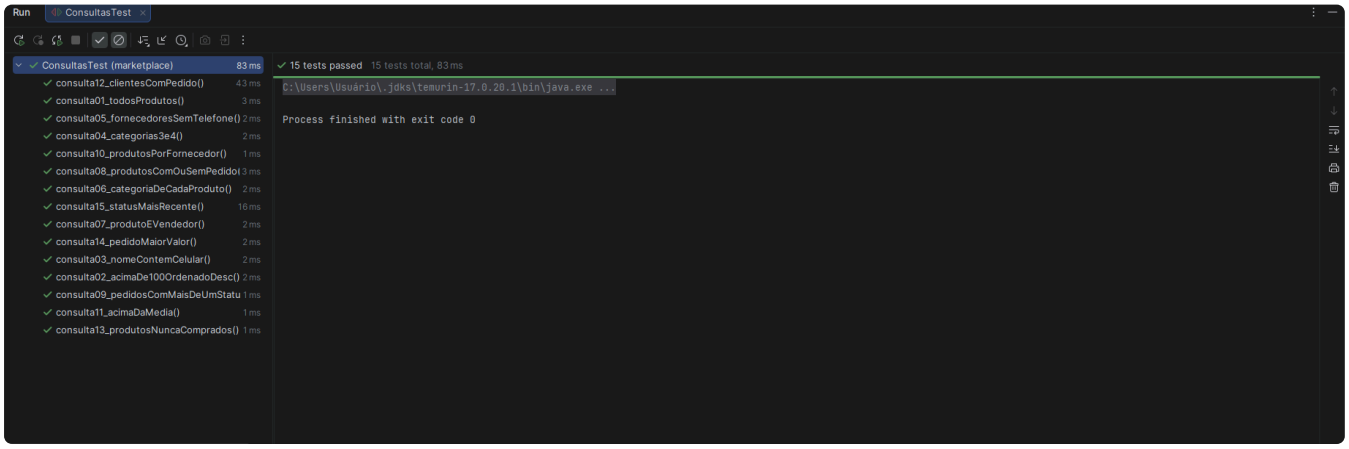


Figura 4 — Execução do BugsConhecidosTest: os 4 testes falharam, como esperado.

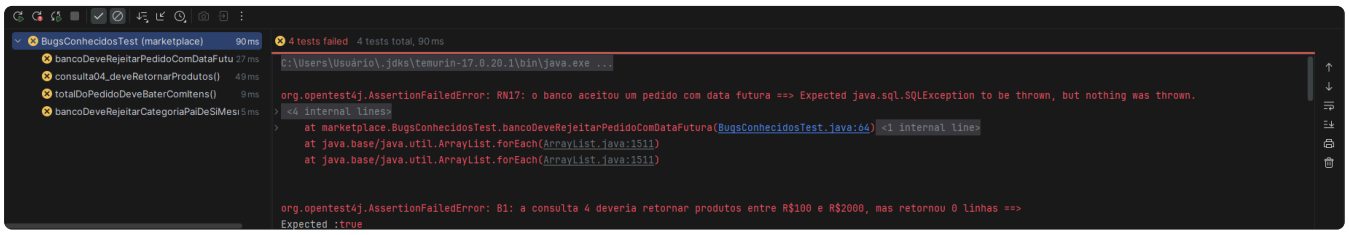


Figura 5 — BUG-U-01: a consulta 4 retornou 0 linhas.



Figura 6 — BUG-U-02: o banco aceitou um pedido com data futura.

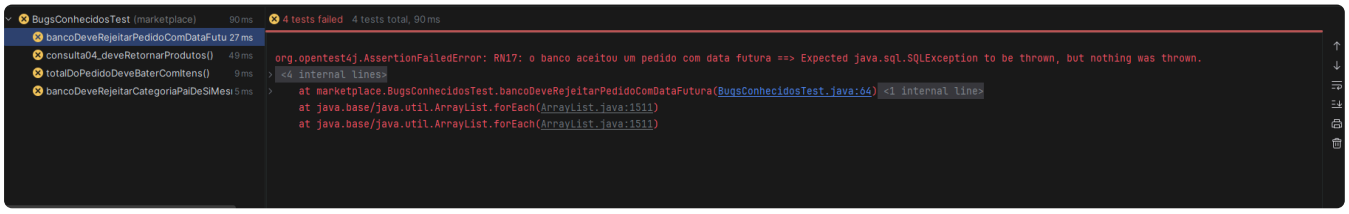


Figura 7 — BUG-U-03: o banco aceitou uma categoria ligada a ela mesma.

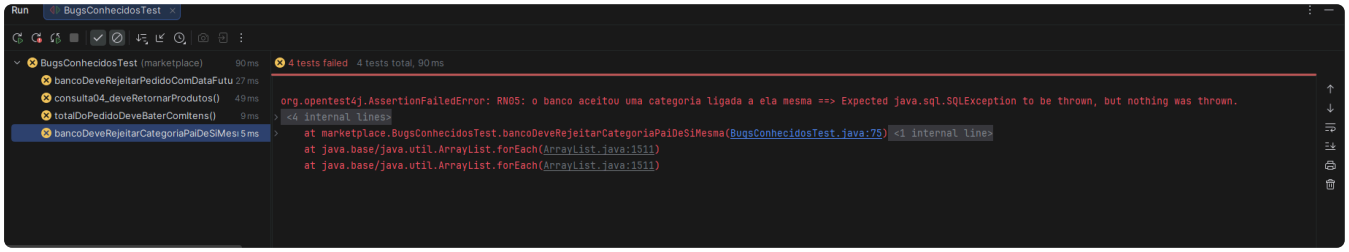
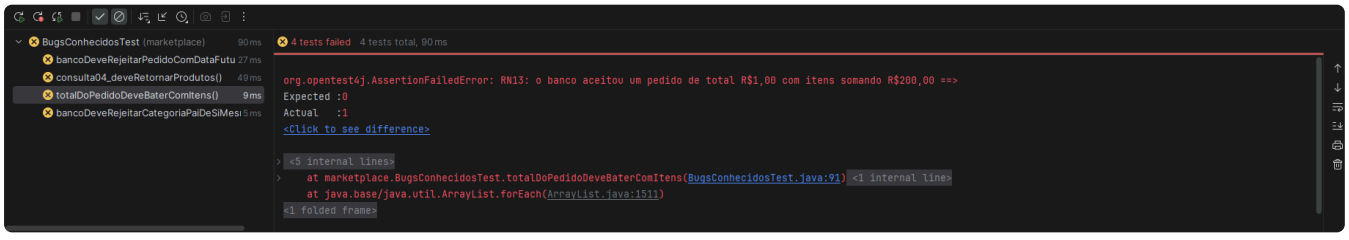


Figura 8 — BUG-U-04: o banco aceitou um pedido de total R$ 1,00 com itens somando R$ 200,00.



Relatório gerado pelo Maven Surefire (comando mvn test, 21/10/2026): 33 testes executados, 4 falhas (todas no BugsConhecidosTest, esperadas), 0 erros e 0 ignorados. O build termina em BUILD FAILURE exclusivamente por causa dessas 4 falhas intencionais. Os arquivos individuais de cada classe ficam em target/surefire-reports.

Classe	Testes	Falhas	Erros	Tempo (Maven)
BugsConhecidosTest	4	4 (esperadas)	0	2,038 s
CargaTest	9	0	0	1,267 s
ConsultasTest	15	0	0	1,140 s
DdlTest	5	0	0	3,799 s
Total	33	4	0	47,6 s (tempo total do build)

Evidências dos testes de API

Execução da coleção no Postman (21/10/2026): 22 requisições e 47 verificações, com 44 aprovadas e 3 falhas, em 3 s 513 ms e sem erros de conexão.

Figura 9 — API-01 a API-11: todas as verificações aprovadas.

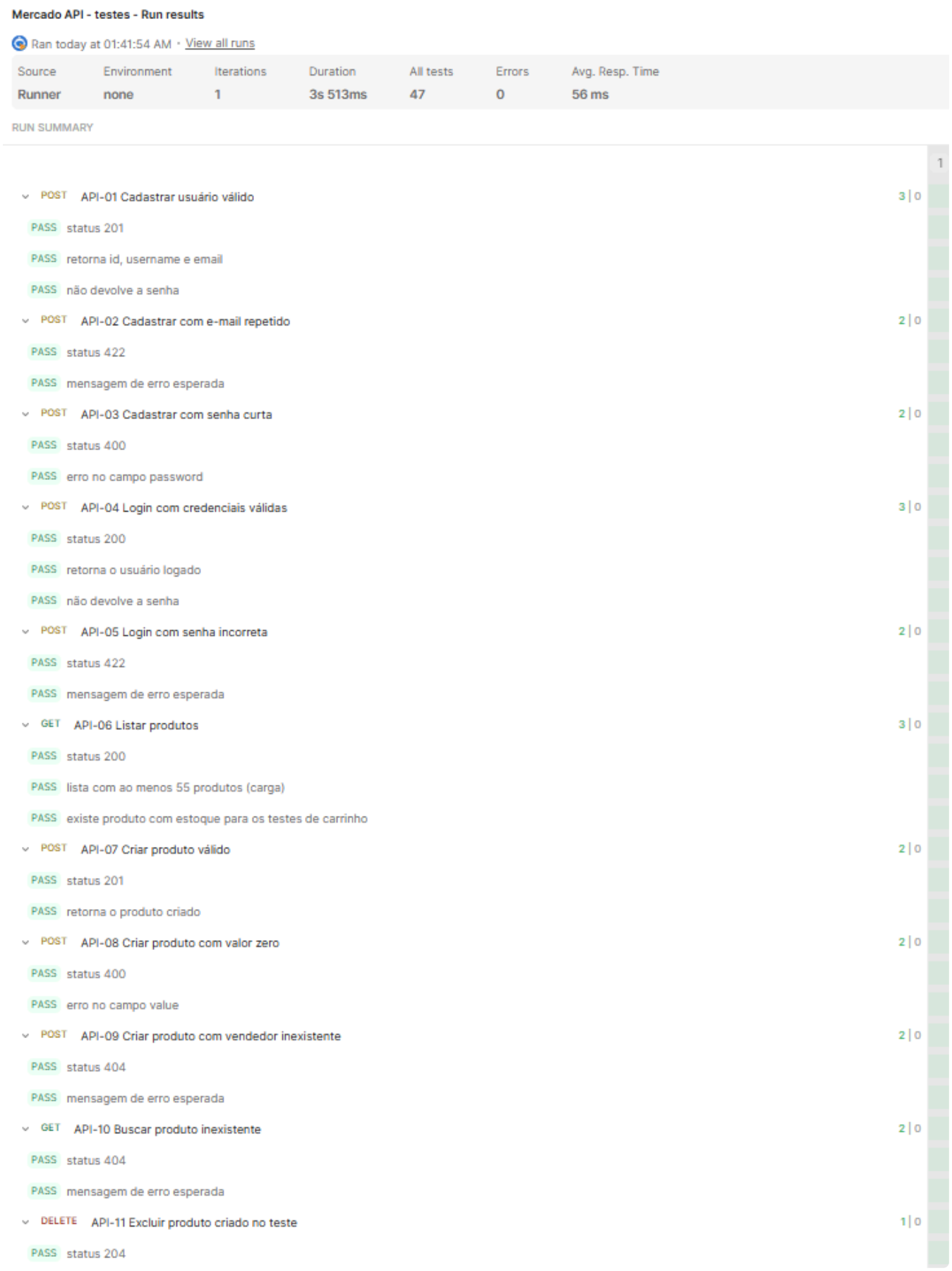


Figura 10 — API-11 a API-22: as falhas aparecem no API-21 e no API-22.

Mercado API - testes - Run results

Ran today at 01:41:54 AM · View all runs

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	none	1	3s 513ms	47	0	56 ms

RUN SUMMARY

1

- PASS mensagem de erro esperada
- DELETE API-11 Excluir produto criado no teste 1 | 0
  - PASS status 204
- POST API-12 Tornar usuário cliente (cria o carrinho) 1 | 0
  - PASS status 201
- GET API-13 Consultar carrinho do cliente 2 | 0
  - PASS status 200
  - PASS carrinho vazio
- POST API-14 Fechar pedido com carrinho vazio 2 | 0
  - PASS status 422
  - PASS mensagem de erro esperada
- POST API-15 Adicionar item ao carrinho 3 | 0
  - PASS status 200
  - PASS carrinho com 1 item e quantidade 2
  - PASS subtotal igual a valor unitário x 2
- POST API-16 Adicionar item acima do estoque 2 | 0
  - PASS status 422
  - PASS mensagem de estoque insuficiente
- POST API-17 Fechar pedido válido 3 | 0
  - PASS status 201
  - PASS pedido aberto com 1 item
  - PASS total igual a valor unitário x 2 (RN13)
- GET API-18 Estoque diminui após o pedido 2 | 0
  - PASS status 200
  - PASS estoque reduzido em 2
- PATCH API-19 Status inválido (ABERTO para ENTREGUE) 2 | 0
  - PASS status 422
  - PASS mensagem de erro esperada
- PATCH API-20 Status válido (ABERTO para PAGO) 2 | 0
  - PASS status 200
  - PASS status PAGO e histórico com 2 registros
- PATCH API-21 Cancelar pedido pago 0 | 2 ✕
  - FAIL status 200 ✕
  - FAIL status CANCELADO ✕
- GET API-22 Estoque volta após o cancelamento 1 | 1 ✕
  - PASS status 200
  - FAIL estoque restaurado ✕

Evidências do teste unitário do login

Figura 11 — UserServiceLoginTest: 4 testes unitários do login aprovados em 1 s 693 ms (o aviso em vermelho do Mockito é apenas informativo sobre a versão do JDK).



Evidências dos testes E2E

Figura 12 — Execução dos testes E2E no terminal: E2E-01 a E2E-08 aprovados e E2E-09 falhando, com o pedido ainda em "Aguardando pagamento" após o cancelamento.

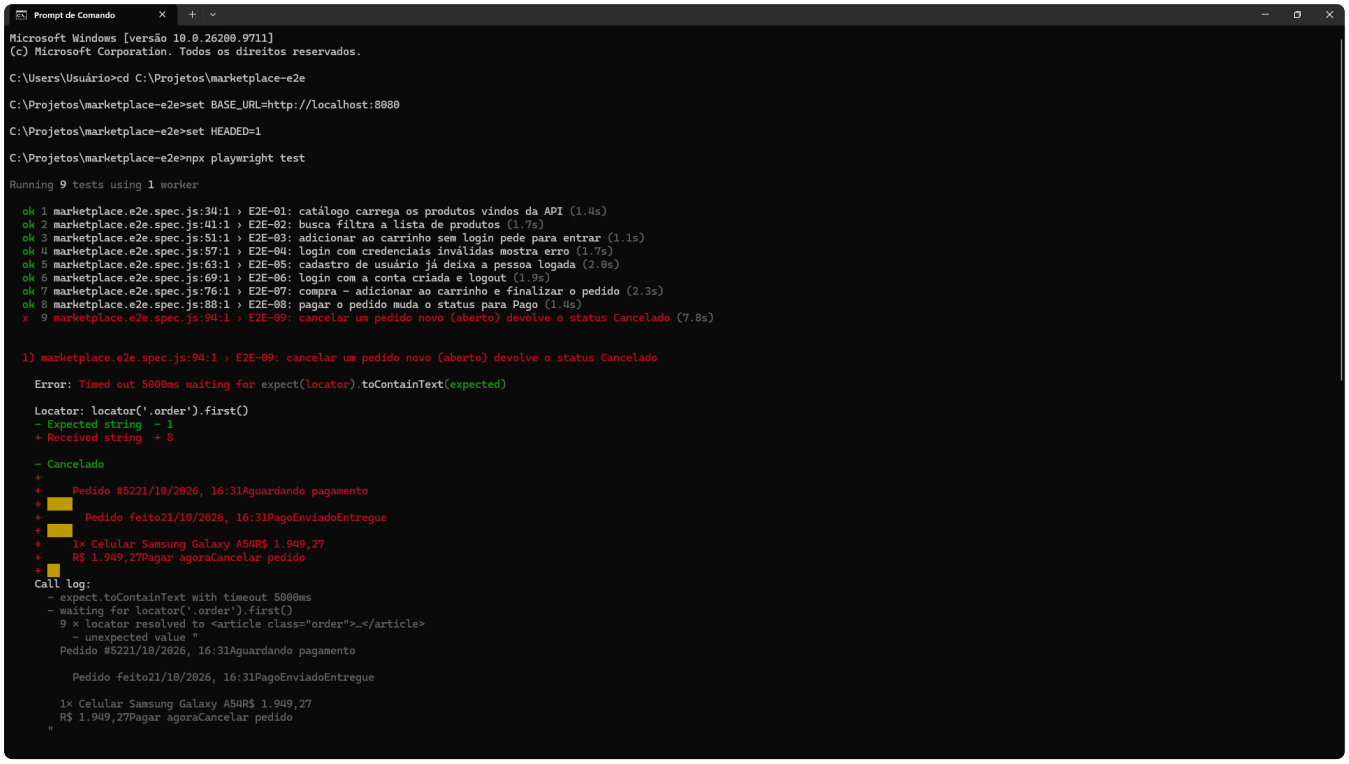
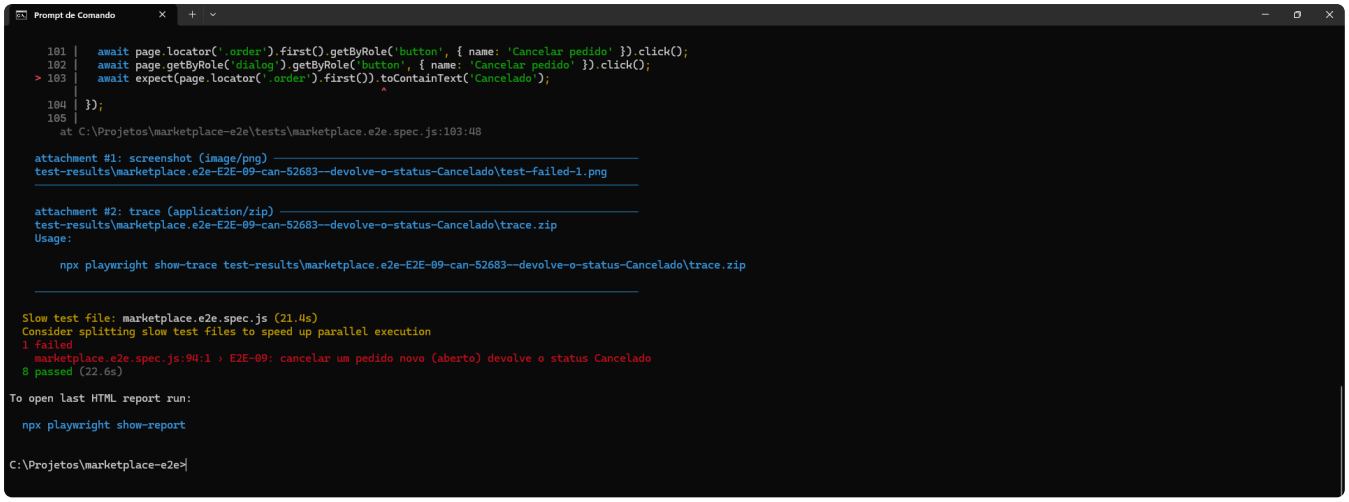


Figura 13 — Resumo da execução E2E: 8 aprovados e 1 falha em 22,6 s, com o caminho do print e do trace gerados pelo Playwright.



6. Bugs e falhas identificados

Nenhum dos 29 testes de verificação do banco falhou; na coleção de API, 3 verificações falharam (achado B5, Figura 10), e na interface 1 dos 9 testes E2E falhou, no cancelamento de pedido (achado B5, Figura 12); os 4 testes de BugsConhecidosTest falham de propósito e comprovam os achados B1 e B4 (Figuras 4 a 8). Os pontos abaixo foram identificados durante a análise dos scripts e a definição dos resultados esperados.

ID	Descrição	Onde	Impacto	Sugestão
B1	A consulta 4 retorna 0 linhas. As categorias 3 e 4 (Alimentos e Vestuário) são categorias raiz, e os produtos estão nas subcategorias 5 a 14.	03_consultas.sql, consulta 4	A consulta não responde ao enunciado	Filtrar pelas subcategorias desejadas (por exemplo 5 e 8) ou considerar id_category_parent. O teste CONS-U-04 registra o comportamento atual e o BUG-U-01 falha comprovando o problema (Figura 5).

ID	Descrição	Onde	Impacto	Sugestão
B2	Comentários da consulta desatualizados em relação à carga atual: consulta 2 (diz que aparecem só celular e panela, mas retorna 37 produtos); consulta 5 (cita Elias sem telefone, mas Elias Comercio tem telefone); consulta 11 (diz média 810 e só o celular, mas a média é 821,57 e são 17 produtos); consulta 13 (diz que só a panela nunca foi comprada, mas são 35 produtos); consulta 14 (atribui o maior pedido ao celular, mas o pedido 4 é composto principalmente por 4 notebooks e 4 celulares).	03_consultas.sql	Documentação inconsistente com os dados	Atualizar os comentários com os valores reais.

ID	Descrição	Onde	Impacto	Sugestão
B3	Comentário da carga informa que apenas o fornecedor 16 fica sem produtos, mas 10 dos 16 fornecedores não têm produto vinculado (Elias Comercio, TechImport Ltda, Alimentos Brasil, Moda Sul Confeccoes, Calcados Nordeste, Utilcasa Distribuicao, MoveisBem Fabrica, Acessorios Tech, Norte Suprimentos e Sabor Distribuidora).	02_carga.sql, comentário de tb_product_supplier	Documentação inconsistente com os dados	Corrigir o comentário ou ajustar a carga.
B4	Observação: conforme os comentários do próprio DDL, as regras RN11, RN12, RN13 e RN17 e a proibição de categoria ligada a ela mesma são controladas pela aplicação, não pelo banco. Os testes CARGA-U-02, 04 e 05 verificam apenas os dados da carga; o banco, por si só, aceita registros fora dessas regras, como comprovam os testes BUG-U-02, BUG-U-03 e BUG-U-04 (Figuras 6 a 8).	01_ddl.sql	Risco de dados inconsistentes se a aplicação não validar	Garantir validação na aplicação ou avaliar triggers.

ID	Descrição	Onde	Impacto	Sugestão
B5	Ao cancelar um pedido pago logo após o pagamento (API-21), a API não retornou 200 com status CANCELADO, e o estoque não foi restaurado (API-22). Causa provável: o backend grava a data de cada status truncada em segundos e a chave primária de tb_order_status é (id_order, date_status), então duas mudanças de status do mesmo pedido no mesmo segundo colidem. O Postman enviou PAGO e CANCELADO com poucos milissegundos de diferença. O mesmo comportamento apareceu na interface (E2E-09, Figura 12): cancelar um pedido logo após criá-lo não mudou o status, que continuou "Aguardando pagamento"; a mensagem de erro exibida não foi registrada. A confirmar repetindo o cancelamento alguns segundos depois da criação do pedido.	Backend Mercado (PurchaseOrder.addStatus) e chave de tb_order_status em 01_ddl.sql	Mudanças de status em sequência rápida podem falhar (Figura 10)	Usar precisão de milissegundos em date_status (DATETIME(3)) ou incluir um número de sequência na chave. Em testes automatizados, aguardar mais de 1 segundo entre mudanças de status do mesmo pedido.

7. Conclusão

Dos 33 testes de banco, 29 foram aprovados e 4 (BugsConhecidosTest) falharam de propósito para evidenciar bugs. Na coleção de API (Postman), 44 das 47 verificações foram aprovadas e 3 falharam, todas no cancelamento de pedido pago (achado B5). O DDL rejeita os dados inválidos testados (e-mail duplicado, valor zero, status inválido, exclusão de vendedor com produtos) e aplica a exclusão em cascata esperada. A carga é consistente com as regras de negócio verificadas, e as 15 consultas retornam os resultados esperados para os dados atuais. As falhas encontradas dizem respeito à adequação da consulta 4 e a comentários desatualizados nos scripts (B1 a B3), além da observação de risco sobre regras controladas pela aplicação (B4) e da falha ao cancelar pedidos em sequência rápida (B5).

Os 4 testes unitários do login (UserServiceLoginTest) foram aprovados: o login aceita credenciais válidas, rejeita senha errada e e-mail inexistente com a mesma mensagem, e a senha é guardada com hash BCrypt.

Nos testes de interface (E2E), 8 dos 9 casos foram aprovados: catálogo, busca, login, cadastro, compra e pagamento funcionam na tela. O E2E-09 falhou ao cancelar um pedido recém-criado, o mesmo problema observado na API (B5).

8. Como reproduzir

1. Instalar o JDK 17 e o MySQL Server 8.x, e ajustar usuário e senha na classe DbHelper.
2. Abrir o projeto marketplace-testes no IntelliJ e aguardar o Maven carregar as dependências.
3. Executar as classes DdlTest, CargaTest, ConsultasTest e BugsConhecidosTest (esta deve falhar nos 4 testes), uma por vez (cada uma recria o banco marketplace_db).
4. Opcionalmente, executar mvn test e consultar os relatórios em target/surefire-reports.

Testes de API: com o banco carregado (CargaTest), subir o backend Mercado (Java 21, variáveis DB_USERNAME, DB_PASSWORD e SERVER_PORT=8081), importar a coleção Mercado.postman_collection.json no Postman e executar a coleção inteira (Run collection).

Teste unitário do login: no projeto Mercado, colocar UserServiceLoginTest.java em src/test/java/com/ucb/market/services e executar a classe no IntelliJ (não precisa de banco).

Testes E2E: com Node.js instalado, na pasta marketplace-e2e executar npm install e npx playwright install chromium; com o banco carregado e o Mercado rodando, executar npx playwright test (definir BASE_URL se a porta não for 8081) e abrir o relatório com npx playwright show-report.